

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

6.Plots and Interpretation

]



Figure 1: Sample Images from the Fashion-MNIST Dataset

Interpretation: The dataset has these sample images respectively. We shall work on these sample images to identify if these is a shirt,pant etc and respectively.

0.1 Tensor Shapes Before and After Preprocessing

.. DATA PREPROCESSING

Original Dataset Shapes
x_train : (60000, 28, 28)
y_train : (60000,)
x_test : (10000, 28, 28)
y_test : (10000,)

...
After Flattening
x_train_flat : (60000, 784)
x_test_flat : (10000, 784)

(a) Before Preprocessing

(b) After Preprocessing

Figure 2: Tensor Shapes Before and After Preprocessing

Interpretation:

The left image displays the shapes before preprocessing and the right image displays the shapes after preprocessing.

0.2 Model Summary

```
# Display Model Architecture
print("MODEL ARCHITECTURE")
mlp_model.summary()
```

... MODEL ARCHITECTURE
Model: "Fashion_MNIST_MLP"

Layer (type)	Output Shape	Param #
Hidden_Layer_1 (Dense)	(None, 128)	100,480
Hidden_Layer_2 (Dense)	(None, 64)	8,256
Output_Layer (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

Figure 3: Summary of the Baseline Multi-Layer Perceptron (MLP) Model

Interpretation:

The model summary gives architecture of the neural networks, It tells us how many neurons are present in the input layer, hidden layer and output layer respectively.

0.3 Class Distribution

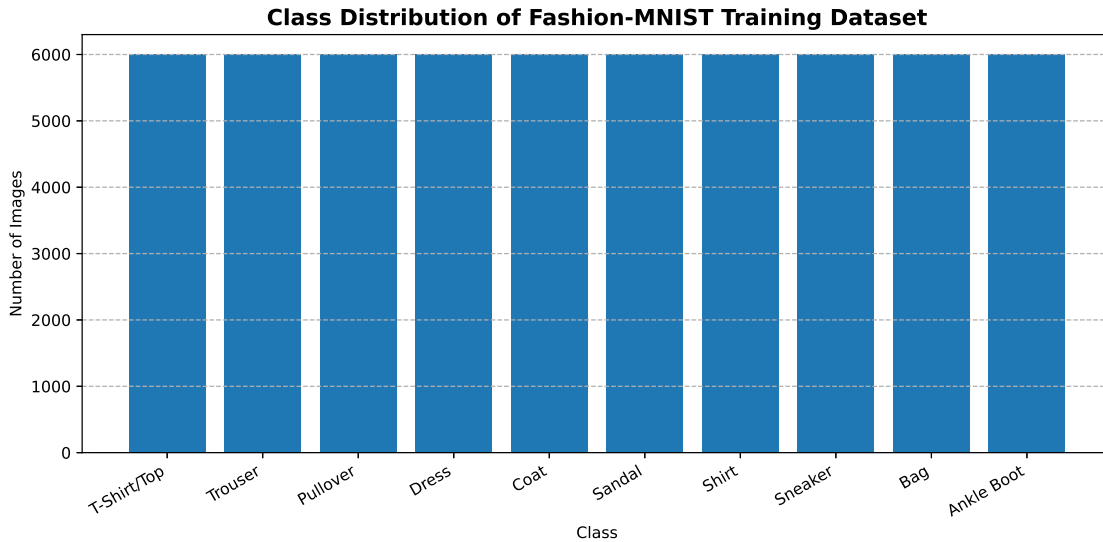


Figure 4: Class Distribution of the Fashion-MNIST Dataset

Interpretation: This plots tells us how many sample are there in each class. All have 6000 samples

0.4 Training Accuracy vs. Epoch

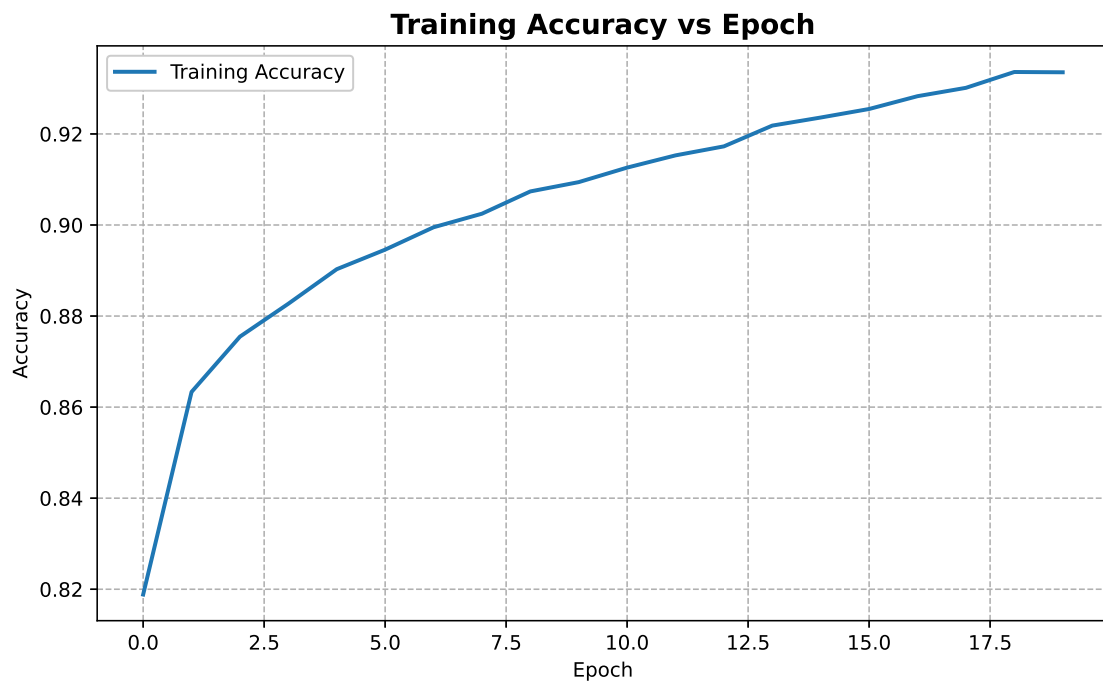


Figure 5: Training Accuracy versus Epoch

Interpretation: The training accuracy vs epochs plot tells us that the training accuracy increases as the number of epochs increases as the model learns with each epoch and the weights get updated, the errors are reduced.

0.5 Validation Accuracy vs. Epoch

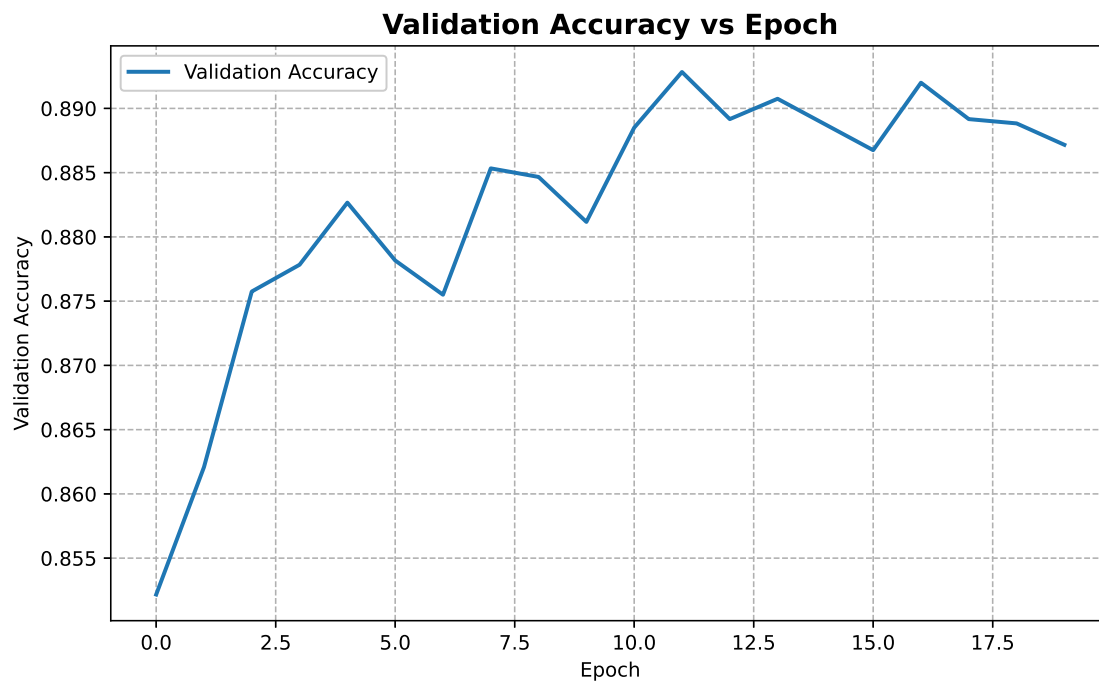


Figure 6: Validation Accuracy versus Epoch

Interpretation: This plot shows that validation accuracy increases as the no of epochs increase. Validation accuracy is the accuracy performed on validation set. Validation dataset means testing the training data so far. The dataset is taken from training dataset only. This prevents the model from overfitting.

0.6 Training Loss vs. Epoch

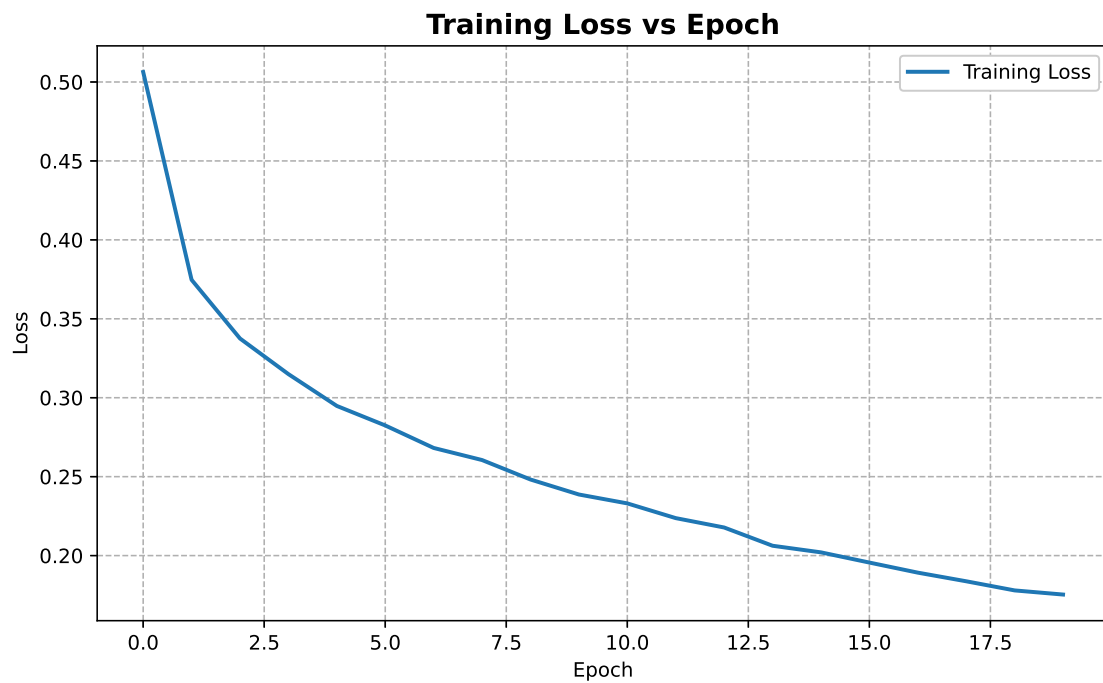


Figure 7: Training Loss versus Epoch

Interpretation: This plot shows that the training loss decrease as the no of epochs increase. As the model learns and updates the weights for each iteration, the error decreases. If the error increase then we should change the parameters to something else.

0.7 Validation Loss vs. Epoch

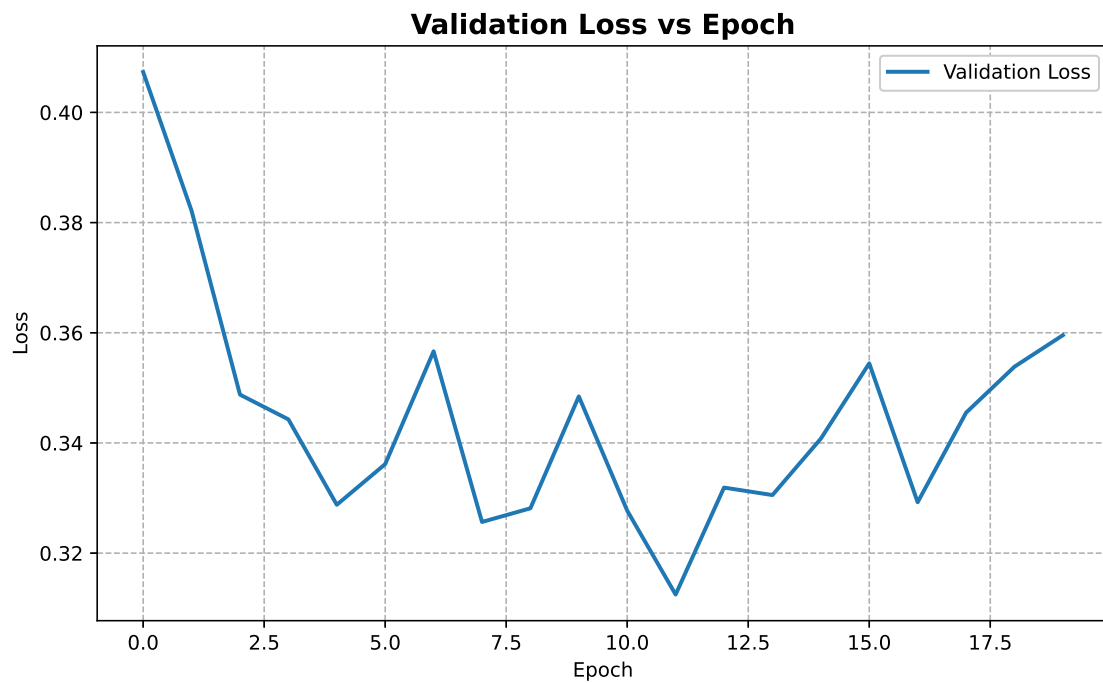


Figure 8: Validation Loss versus Epoch

Interpretation: The validation loss also tends to decrease ,since we test the training data on the validation set sometime the loss necessarily doesn't contionuously decrease it may increase also.Training the data with validation dataset help us tune the hyperparameters and prevents overfitting.

0.8 Confusion Matrix

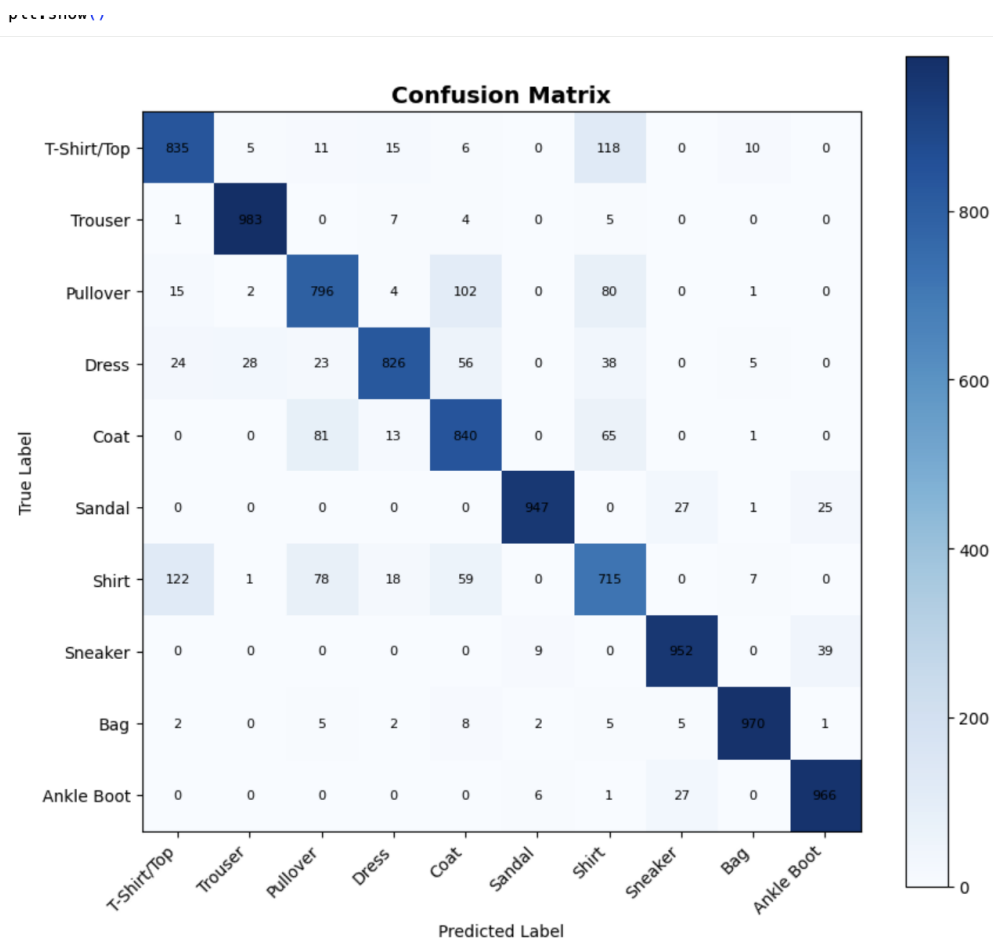


Figure 9: Confusion Matrix of the Baseline Multi-Layer Perceptron (MLP) Model

Interpretation: The confusion matrix tells the classification given by the model if the given label is correct or not, Its one of the performance metrics used to evaluate the results

0.9 Optimized Confusion Matrix

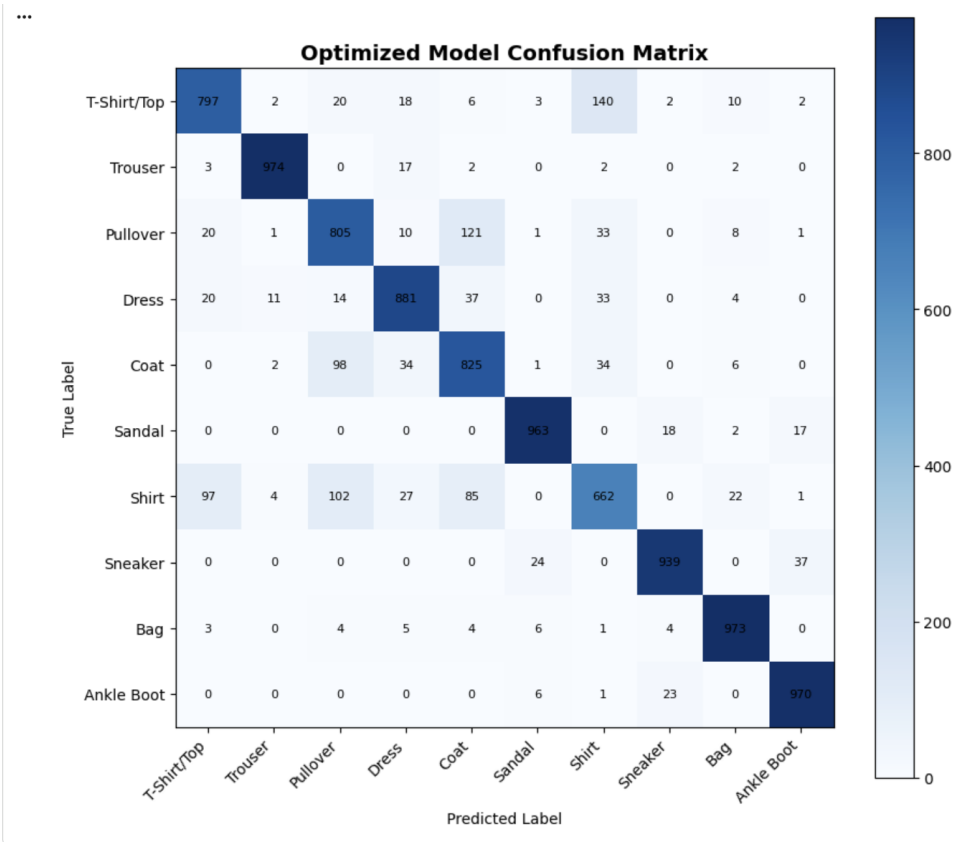


Figure 10: Optimized Confusion Matrix of the Baseline Multi-Layer Perceptron (MLP) Model

Interpretation: Its also a confusion matrix but an optimized version which saves a lot of training time.

0.10 Hyperparameter Search Results

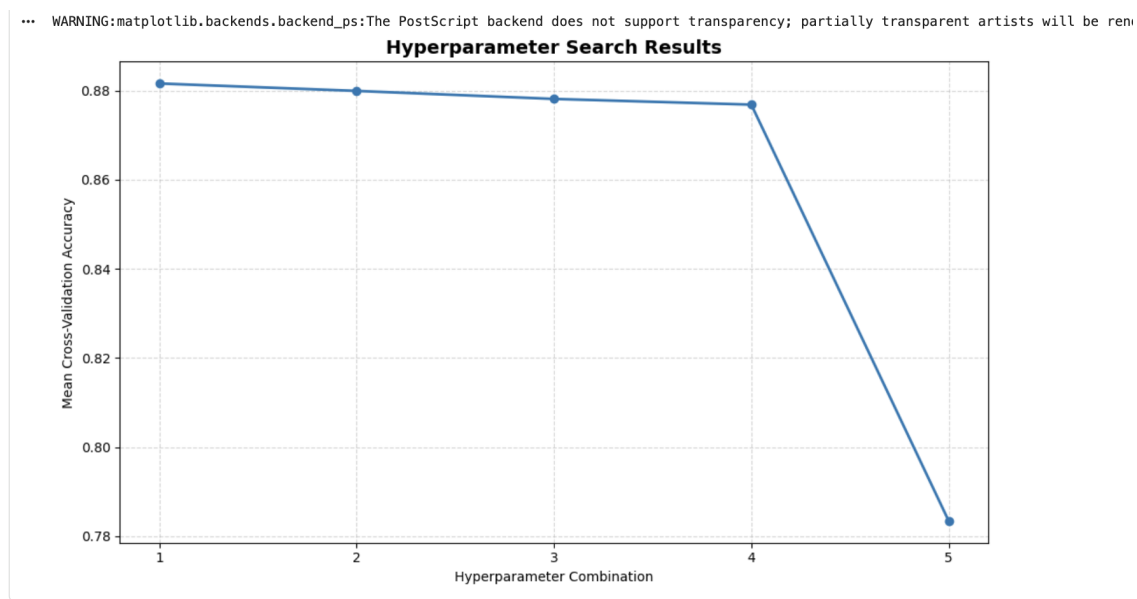


Figure 11: Hyperparameter Search Results using RandomizedSearchCV

Interpretation: This plot shows the hyperparameter combinations across the cvs.

0.11 Best Model Accuracy Comparison

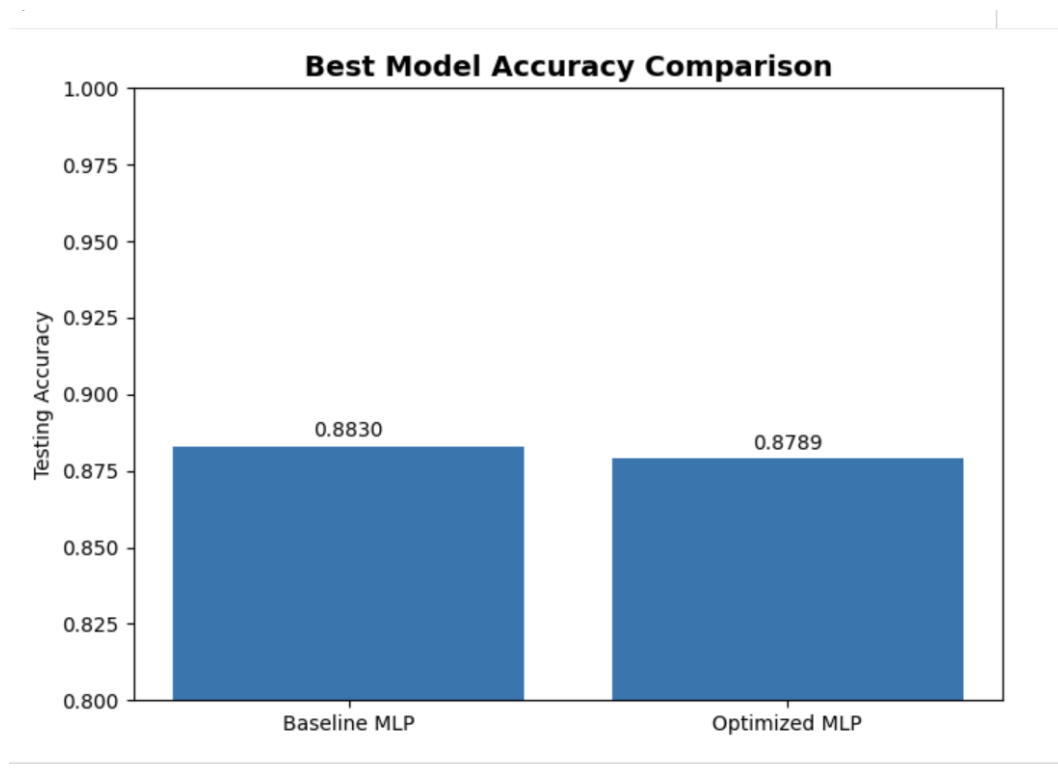


Figure 12: Comparison of Testing Accuracy between the Baseline and Optimized MLP Models

Interpretation: The accuracies are almost similar between baseline MLP and optimized MLP ,The optimized MLP has slightly more accuracy cause the RandomizedSearchCv randomly combines the hyperparamter combinations so it chooses the best model.

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Results

Best Hyperparameters

Hidden Layers	2
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	ADAM
Activation Function	TANH
Epochs	10
Dropout	0.0
Cross-validation Accuracy	0.8856833333333333
Testing Accuracy	0.8854

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8860	0.8854
Precision	0.8870	0.8866
Recall	0.8860	0.8854
F1-score	0.8860	0.8853
Training Time	170.46 seconds	99.61 seconds

9. Discussion

Answer the following:

1. Which optimization method was used? **Answer:**RandomizedSearchCv is the optimization method used.
2. Which hyperparameters were selected? **Answer:**The hyperparamaters used are hidden layers,hidden neurons,learning rate,batch size,epochs,optimizers,activation function and Droupout Rate. 0.3cm
3. Did optimization improve performance? **Answer:**Yes by using optimizers we improved the accuracy performance by little though there no much of a difference.
4. Which hyperparameter had the greatest impact? **Answer:**Learning rate often has greater impact.
5. Compare Grid Search and Randomized Search. **Answer:**Grid Search evaluates every possible hyperparameter combination, whereas Randomized Search evaluates only a random subset of combinations
6. Recommend the best MLP configuration. **Answer:**Depends on the highest accuracy.

10.Conclusion

In this experiment I learned how to implement MLP usig tensor flow and keras,I even learned about Randomized Serach CV.

11. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.
6. Source code:-<https://colab.research.google.com/drive/1jGrfKo8vW6ILTkdydrKiCWekSsLZu0nCK?usp=sharing>
7. Github link:-<https://github.com/Asmita-306/DL-2>